IT314 Software Engineering · Group 13

UML Diagrams

ShopSense: the personalised shopping research agent

Document	UML Diagrams: activity, state, class and sequence
Project	ShopSense: the personalised shopping research agent
Course	IT314 Software Engineering
Team	Group 13 (10 members)
Version	1.0
Date	14 October 2026

This document holds the activity, state, class and sequence diagrams of ShopSense. Class names, fields, types, limits and rules are taken from the stage contracts in *backend/app/contracts* (schema version 1.0.0); behaviour is taken from the product backlog and the product plan. Page 2 explains the notation.

Contents

Figure	Diagram	What it shows	Page
1	Activity diagram	One ShopSense session at a glance	3
2	Activity diagram	Understanding the request	4
3	Activity diagram	Interview and rubric	5
4	Activity diagram	Running the search	6
5	Activity diagram	Results, comparison and purchase	7
6	State diagram	The pre-search session	8
7	State diagram	One interview reply	9
8	State diagram	One search	10
9	State diagram	One stage run	11
10	Class diagram	Pipeline and stages	12
11	Class diagram	The LLM wrapper	13
12	Class diagram	The eleven stages	14
13	Class diagram	Request and criteria	15
14	Class diagram	Interview and rubric	16
15	Class diagram	Sources, claims and products	17
16	Class diagram	The search result	18
17	Sequence diagram	Detecting the request	19
18	Sequence diagram	Preparing the criteria	20
19	Sequence diagram	The interview	21
20	Sequence diagram	Search, part 1: gathering evidence	22
21	Sequence diagram	Search, part 2: deciding and explaining	23
22	Sequence diagram	Search, part 3: prices, saving and the result	24

How to read the diagrams

Activity diagrams (Figures 1 to 5)

Symbol	Meaning
Filled circle	Start of the activity
Circle with a filled centre	End of the activity
Rounded box	An action
Hexagon with a question	A decision; each outgoing arrow is labelled with its answer
Small diamond	A merge point where paths join again
Thick black bar	Fork or join: the actions between two bars run in parallel
Circle with a letter	Connector: the flow continues in the next figure at the same letter
Vertical lanes	Swimlanes: who performs each action (shopper, web app, backend)
Box with a folded corner	A note

State diagrams (Figures 6 to 9)

Symbol	Meaning
Rounded box	A state; a line under the name gives what holds while in it
Arrow label	event [guard] / action: the event that fires the change, the condition, and what is done
Small diamond	A choice point: the next state is chosen by the guards
Box inside a box	A composite state with its own inner states

Class diagrams (Figures 10 to 16)

Symbol	Meaning
+ and -	Public and private members
name: type [0..1]	An optional field (it may be None)
(0..10), (0..1), (0..20)	Allowed range of a value, or the most items a list may hold
a | b | c	The field takes one of the listed literal values
/name	A derived value (a read-only property)
Filled diamond	Composition: the part belongs to the whole; numbers give how many
Hollow diamond	Aggregation: the whole uses parts that exist on their own
Dashed arrow, hollow triangle	Implements an interface
Dashed arrow	Dependency, with a stereotype such as «create» or «call»
Italic name	An interface; a dashed box at its corner lists its type parameters
«stereotype»	The kind of element, such as «boundary», «dataclass» or «exception»

Sequence diagrams (Figures 17 to 22)

Symbol	Meaning
Solid arrow	A call or a request
Dashed arrow	A reply
Thin bar on a lifeline	The participant is active
Arrow from or to the left edge	A request from, or reply to, the web app outside the diagram
alt / else	Alternatives: exactly one branch runs, chosen by its guard
opt	An optional part that runs only when its guard holds
loop	A repeated part; the guard says when it stops
par	Parallel parts that run at the same time

All diagrams use the same names as the code. Words such as Intent, Rubric, CriteriaSet or StageStatus are the contract classes in backend/app/contracts. Stage time limits are in seconds.

Figure 1. Activity diagram: One ShopSense session at a glance

The documented 15-step flow in five phases. Remembered answers (step 5) are pre-filled for the interview (step 4). Figures 2 to 5 show each part in detail.

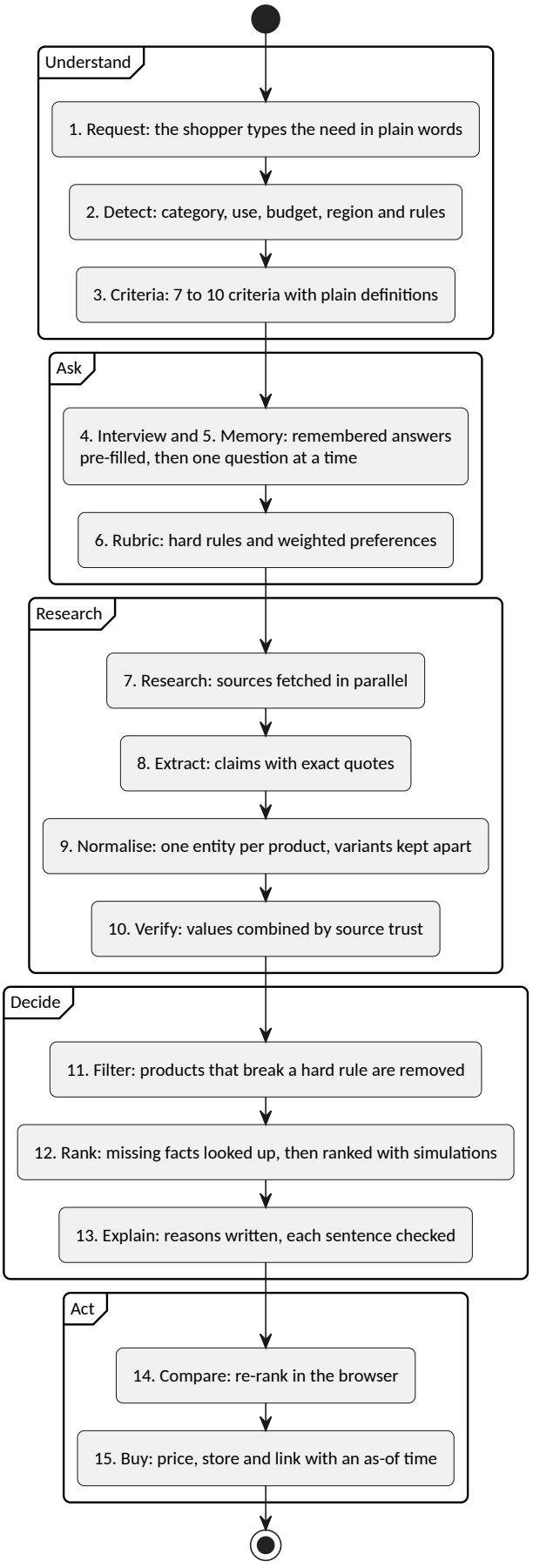


Figure 2. Activity diagram: Understanding the request

Swimlanes show who does each action. The budget is read by plain code and the rest by the LLM wrapper. The app never guesses a category: it asks one follow-up question, then shows example requests. Connector A continues in Figure 3.

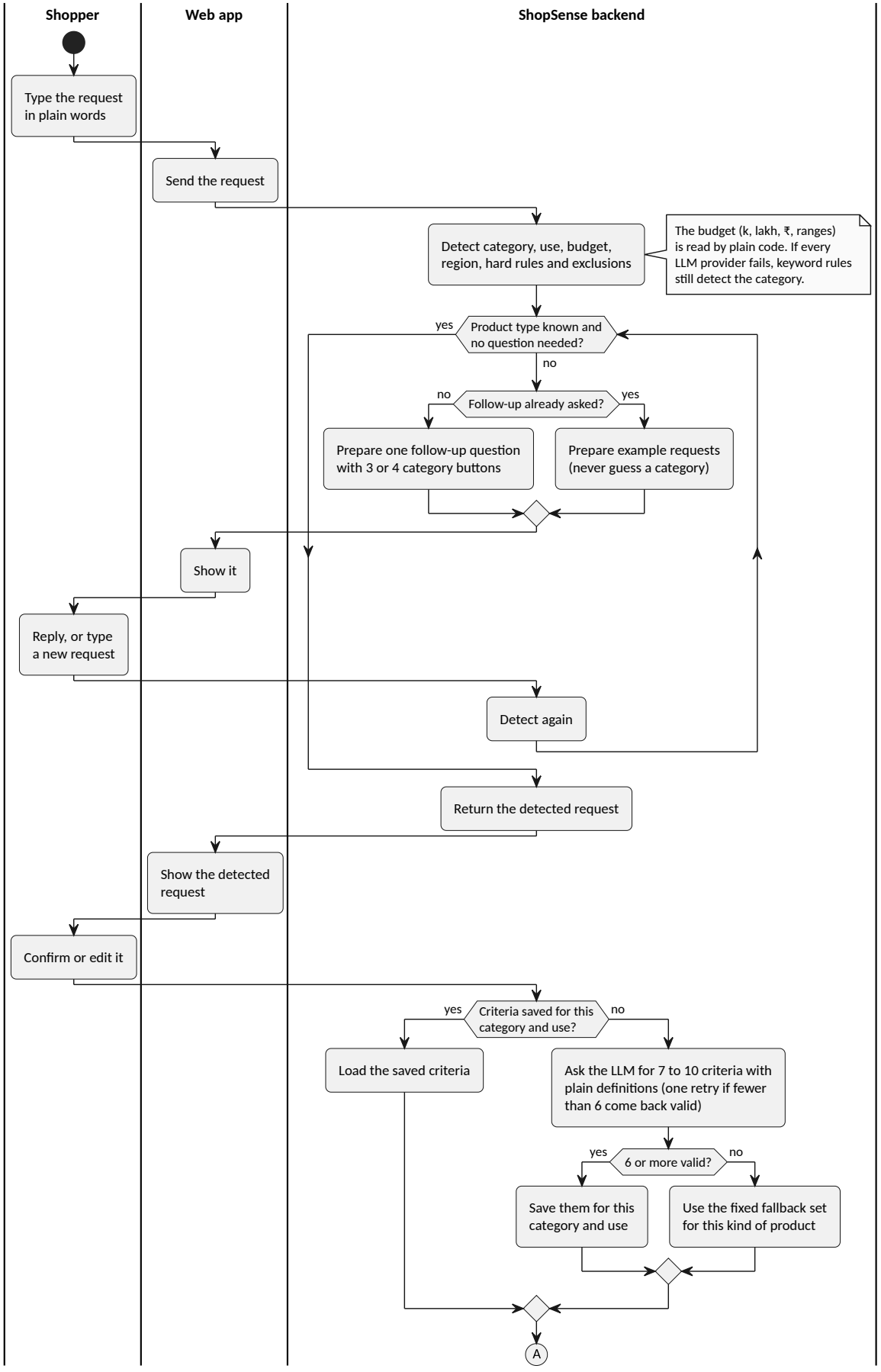


Figure 3. Activity diagram: Interview and rubric

One question per turn. A reply that is not an answer never counts as one (Figure 7 shows every case). The interview ends when the top criteria are settled, no questions are left, or 10 questions have been asked. Connector B continues in Figure 4.

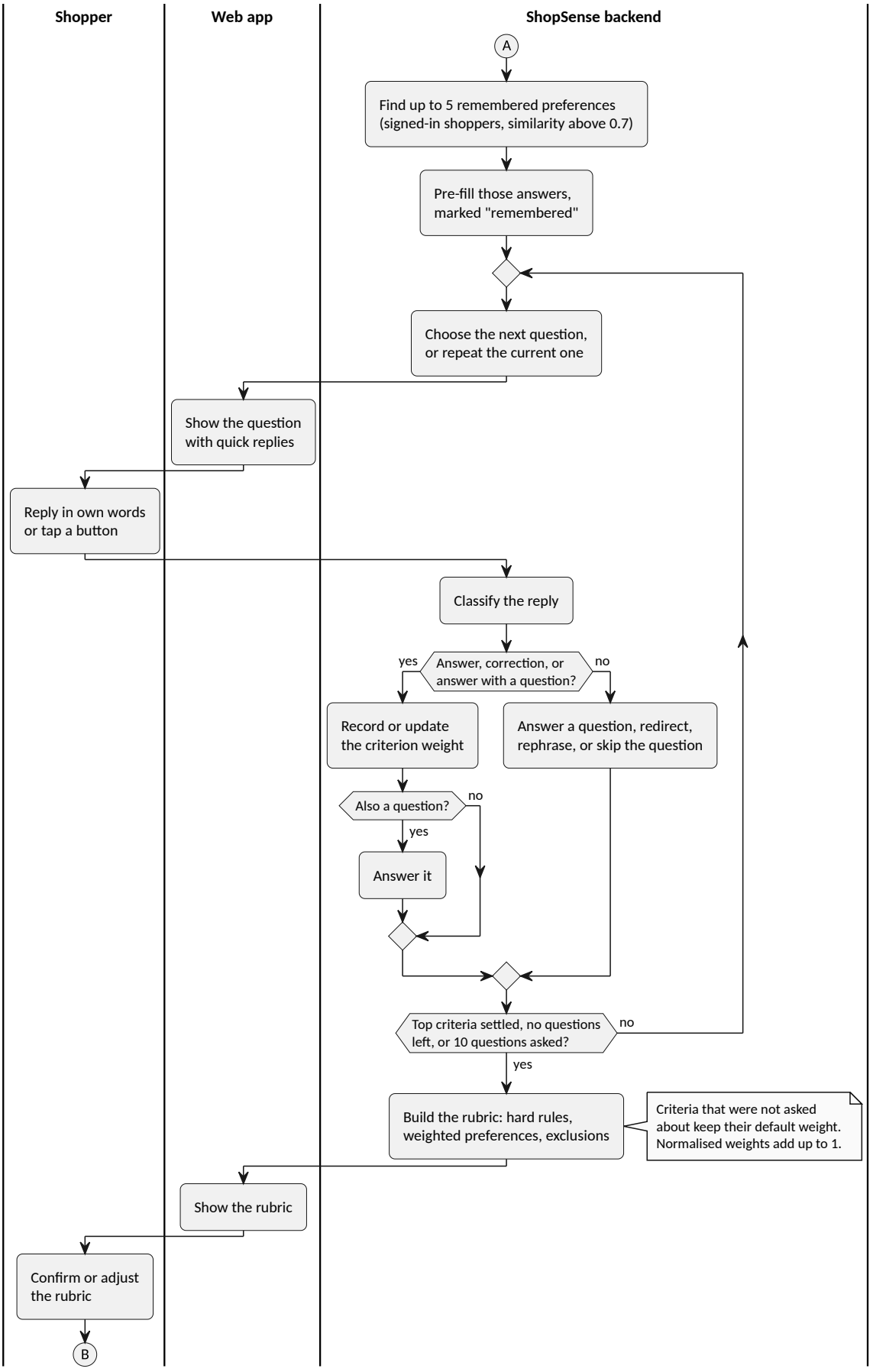


Figure 4. Activity diagram: Running the search

The thick bars split the research into parallel sources and join them again. Every step has a time limit and a fallback, so one failure never stops the search. Filtering and ranking are plain code; the LLM never decides the order. Connector C continues in Figure 5.

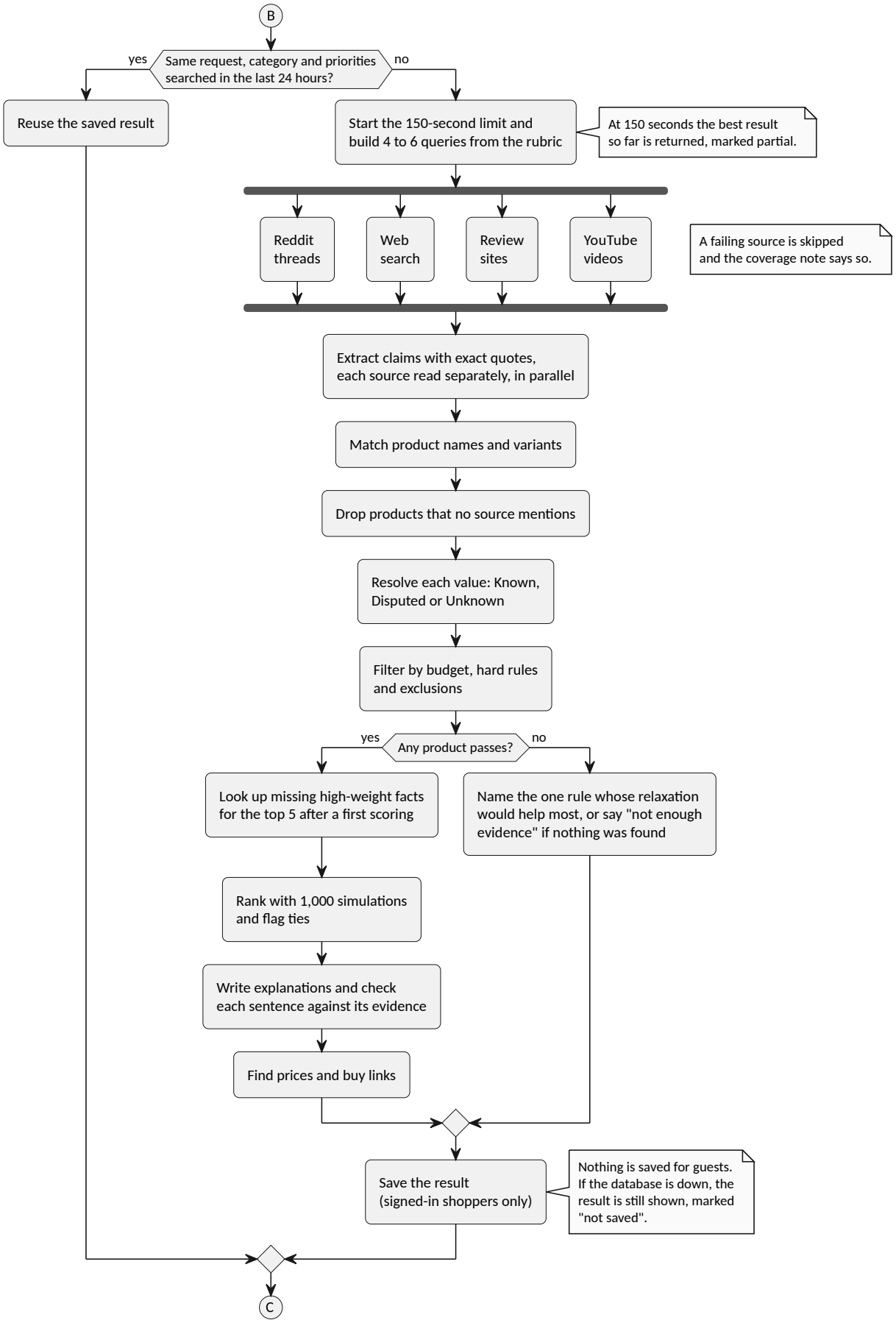


Figure 5. Activity diagram: Results, comparison and purchase

Re-ranking runs in the browser with the same formula as the backend, so no new search is needed. The app opens the store's own page; it never checks out for the shopper.

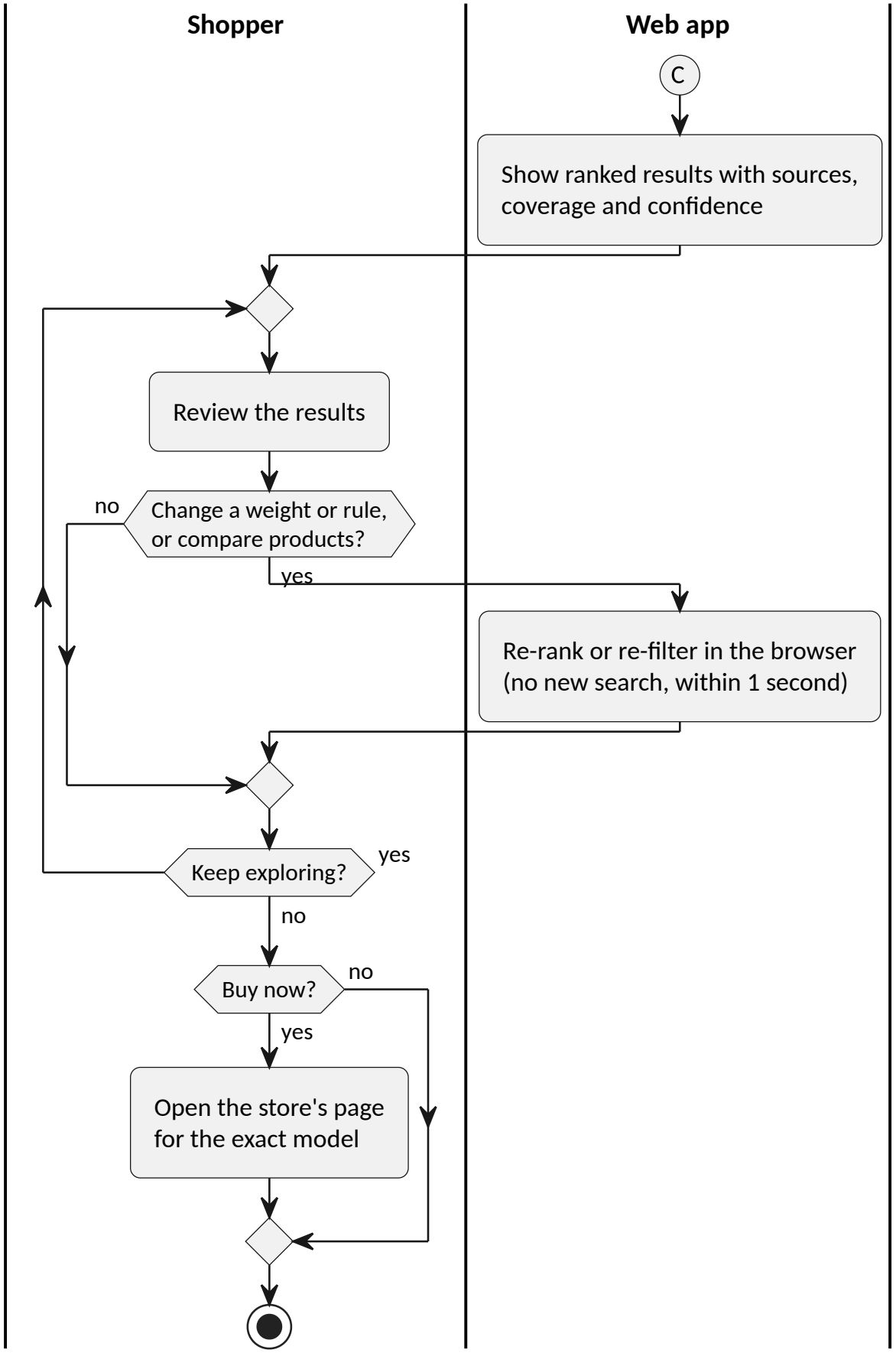


Figure 6. State diagram: The pre-search session

Labels read event [guard] / action. A diamond is a choice point: the next state is picked by the guard in square brackets. The Interviewing state is shown in detail in Figure 7.

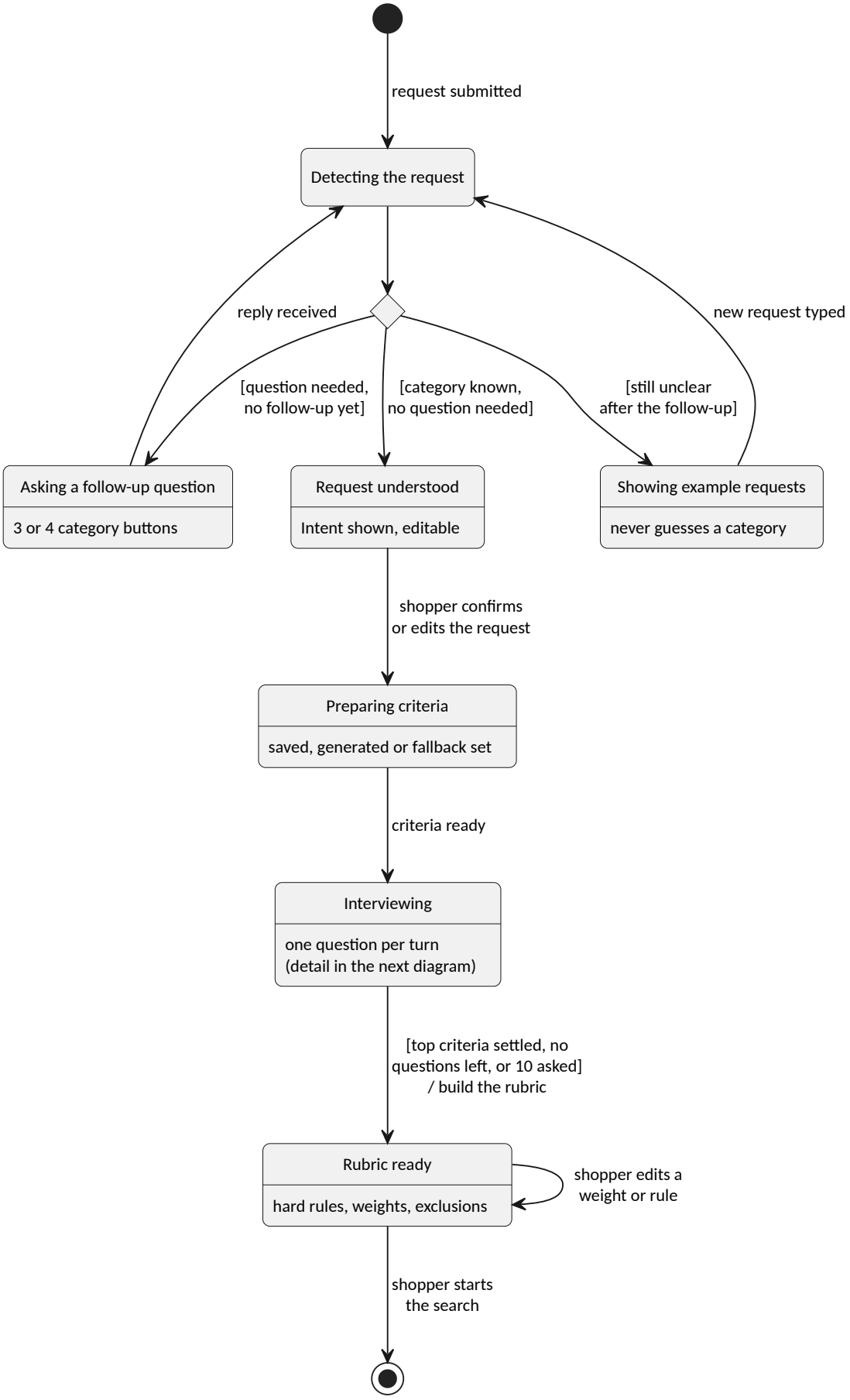


Figure 7. State diagram: One interview reply

How each reply type is handled. Reply text is treated as data only, never as instructions. Questions, redirects and skips never count as answers.

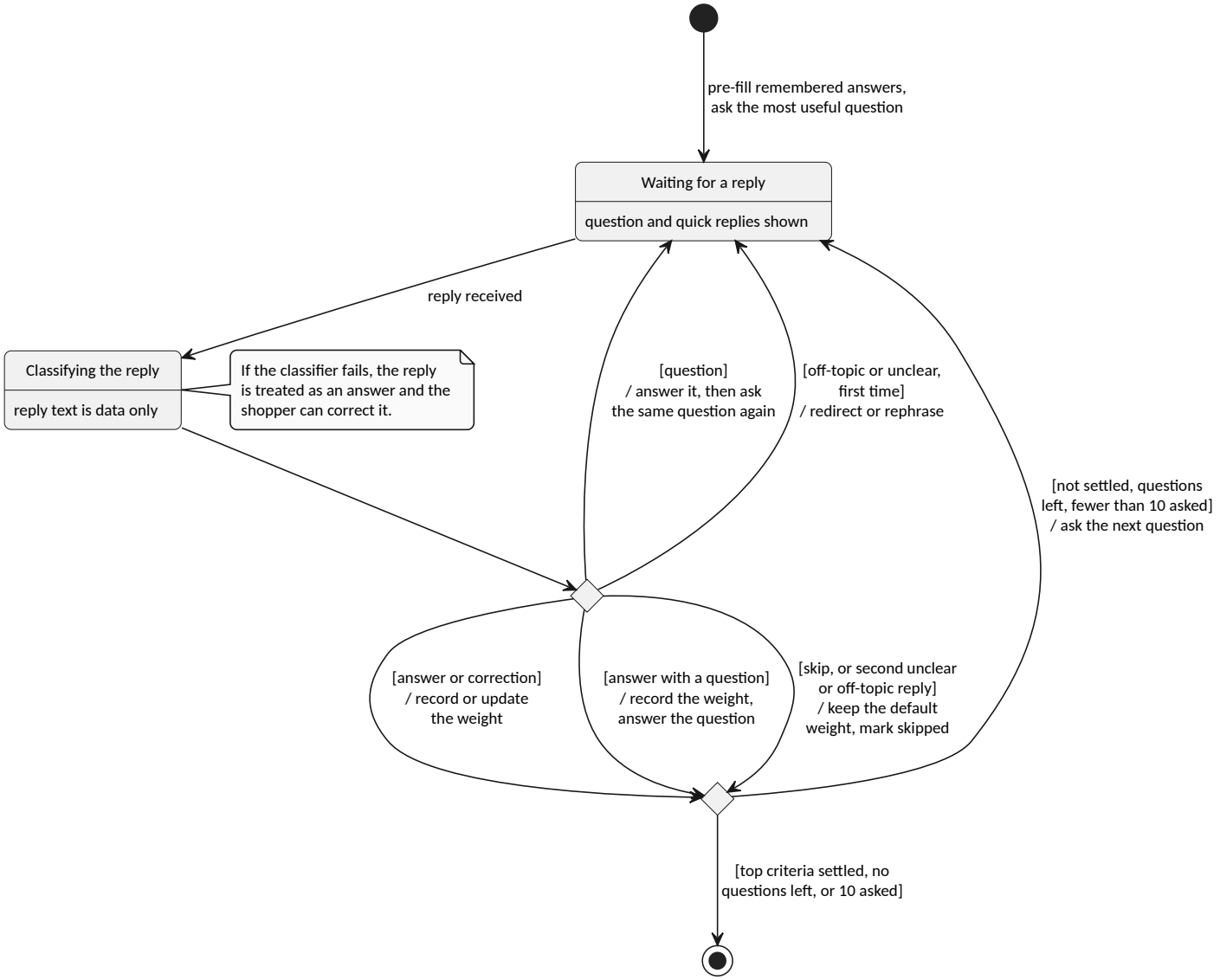


Figure 8. State diagram: One search

From POST /api/search to the result on screen. Inside “Running the pipeline” each stage behaves as in Figure 9. At 150 seconds the best result so far is returned, marked partial.

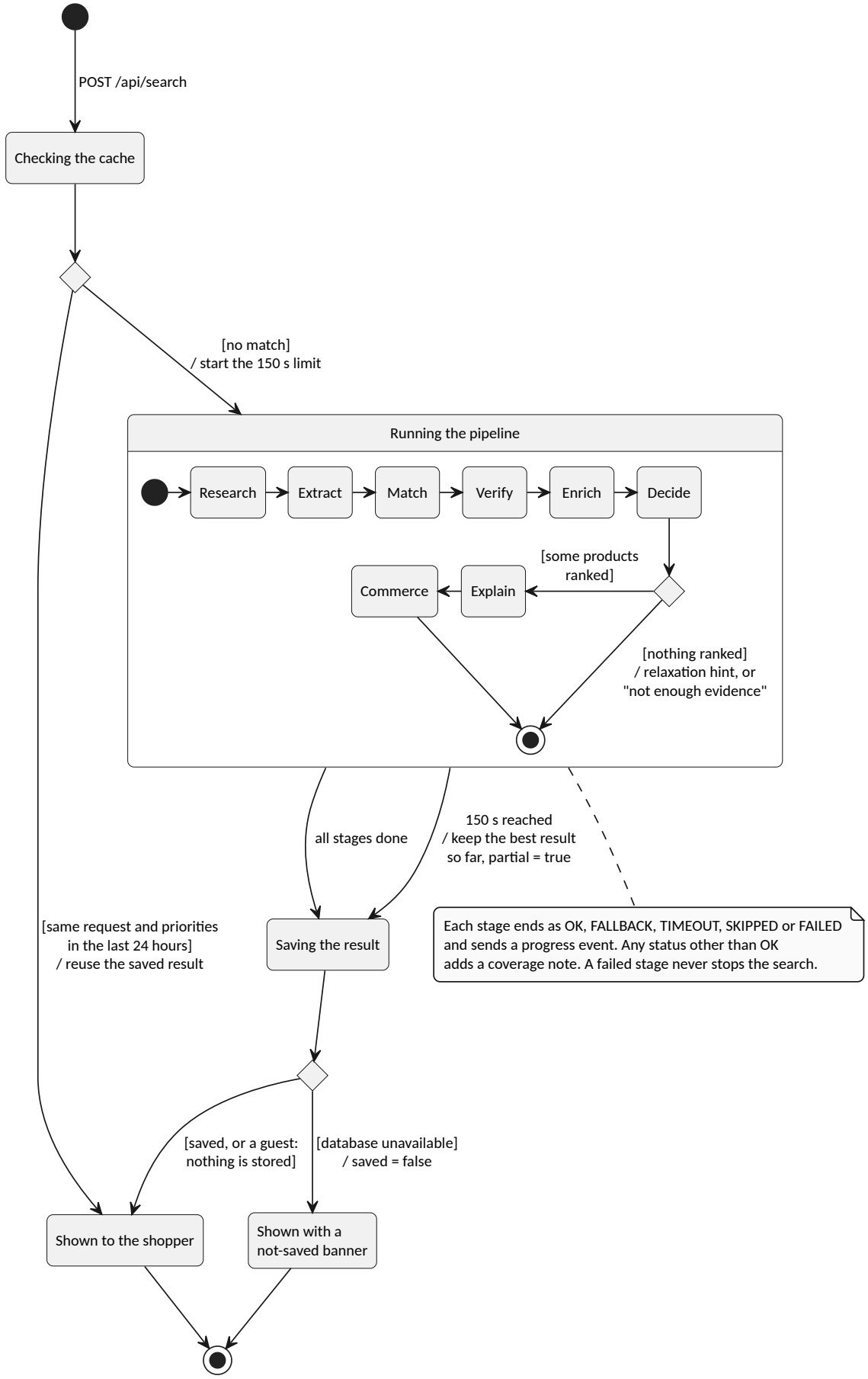


Figure 9. State diagram: One stage run

Every stage ends in exactly one StageStatus, recorded in its StageReport: FALLBACK means the baseline ran, TIMEOUT means the best result so far was kept, and FAILED means even the fallback failed, so the stage produced an empty result.

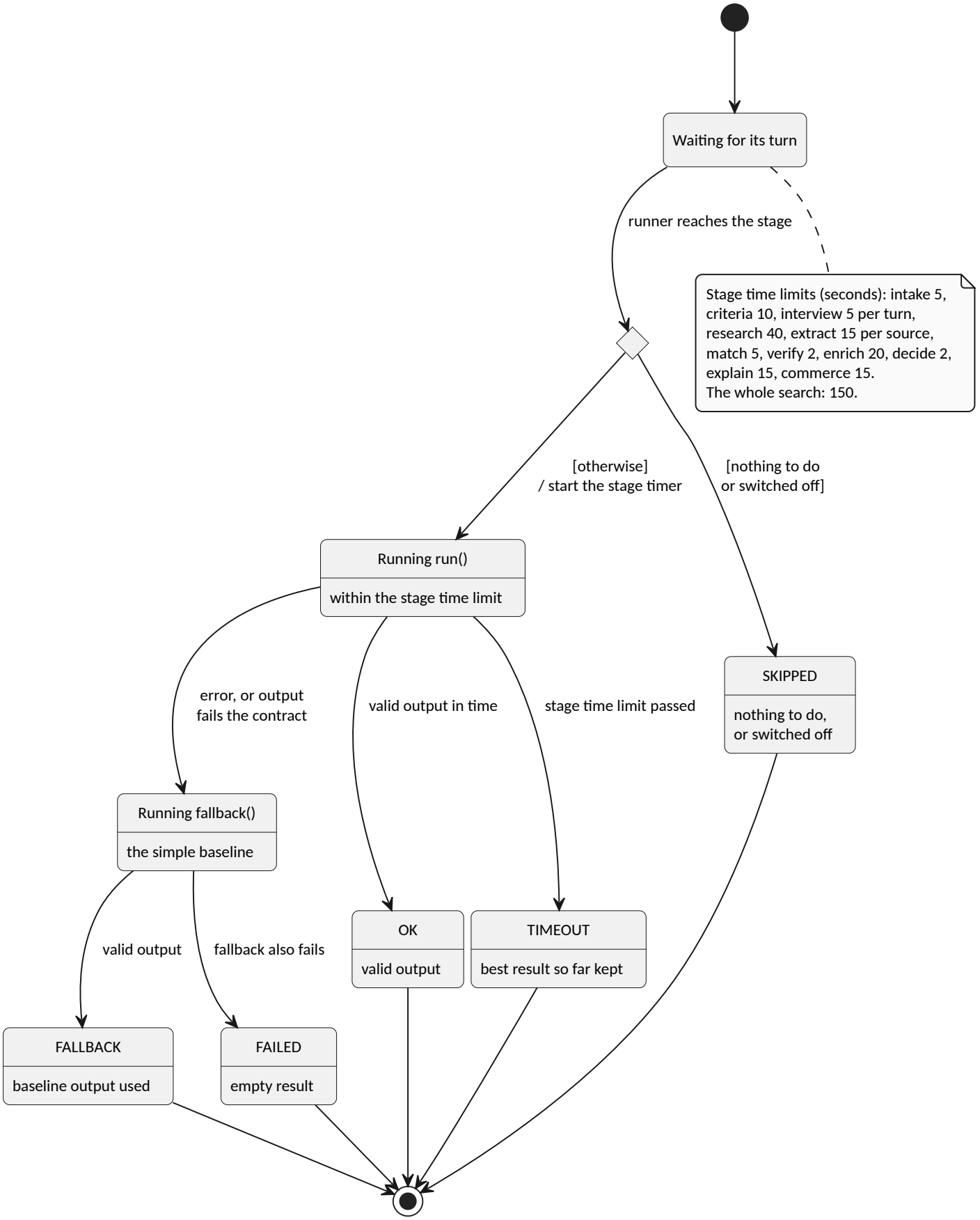


Figure 10. Class diagram: Pipeline and stages

The runner executes the eight search stages, research to commerce, with their time limits. The pre-search routes call IntakeStage, CriteriaStage and InterviewStage directly. ResearchStage shows how every stage receives its dependencies through its constructor.

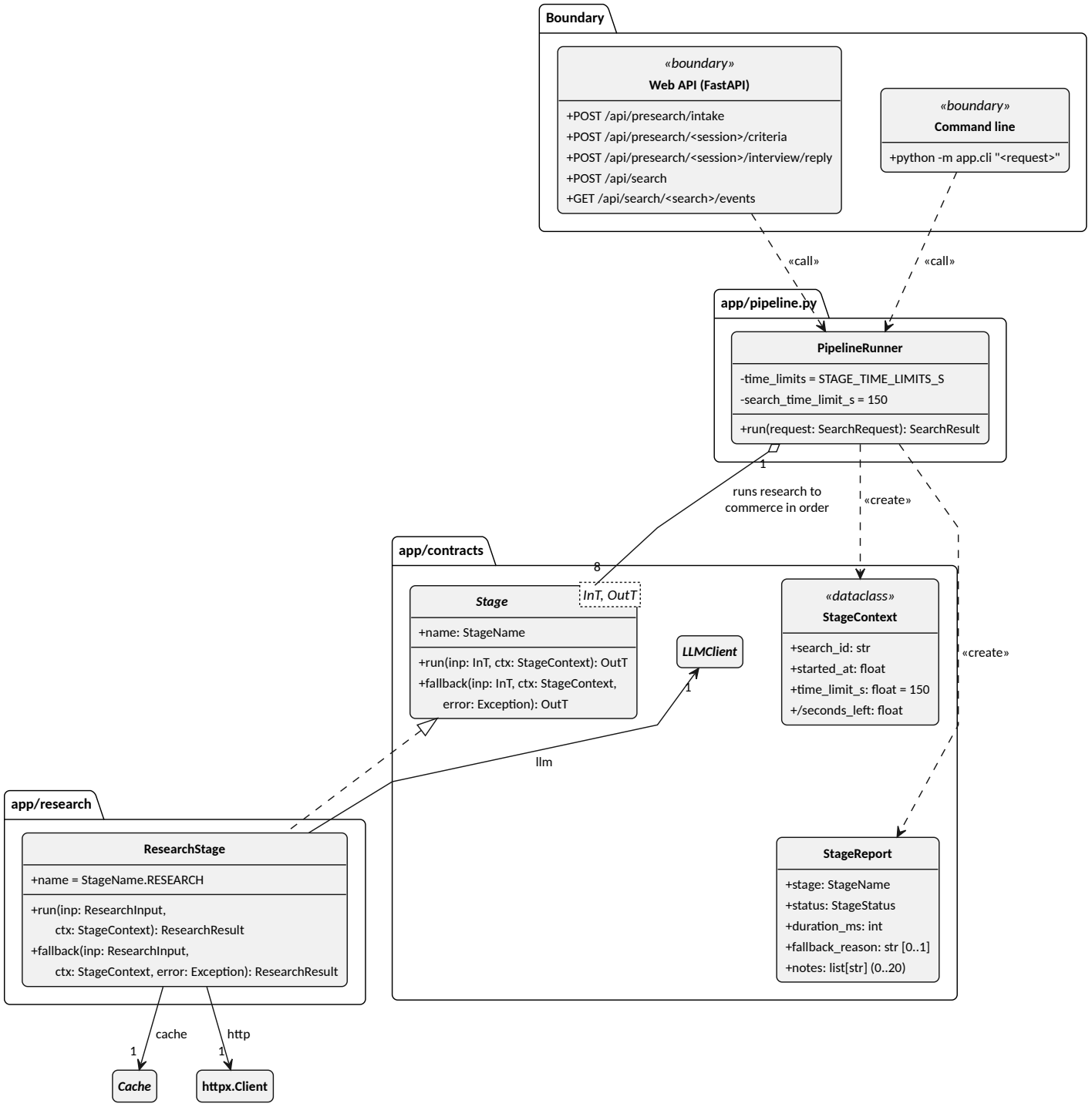


Figure 11. Class diagram: The LLM wrapper

Every LLM call goes through LLMClient.complete. The wrapper fits the input to the model's window (as is, trimmed or chunked), tries Groq, Gemini, OpenRouter and Cerebras in order, validates the reply against the schema, retries invalid output once, and records one LLMCallRecord per call.

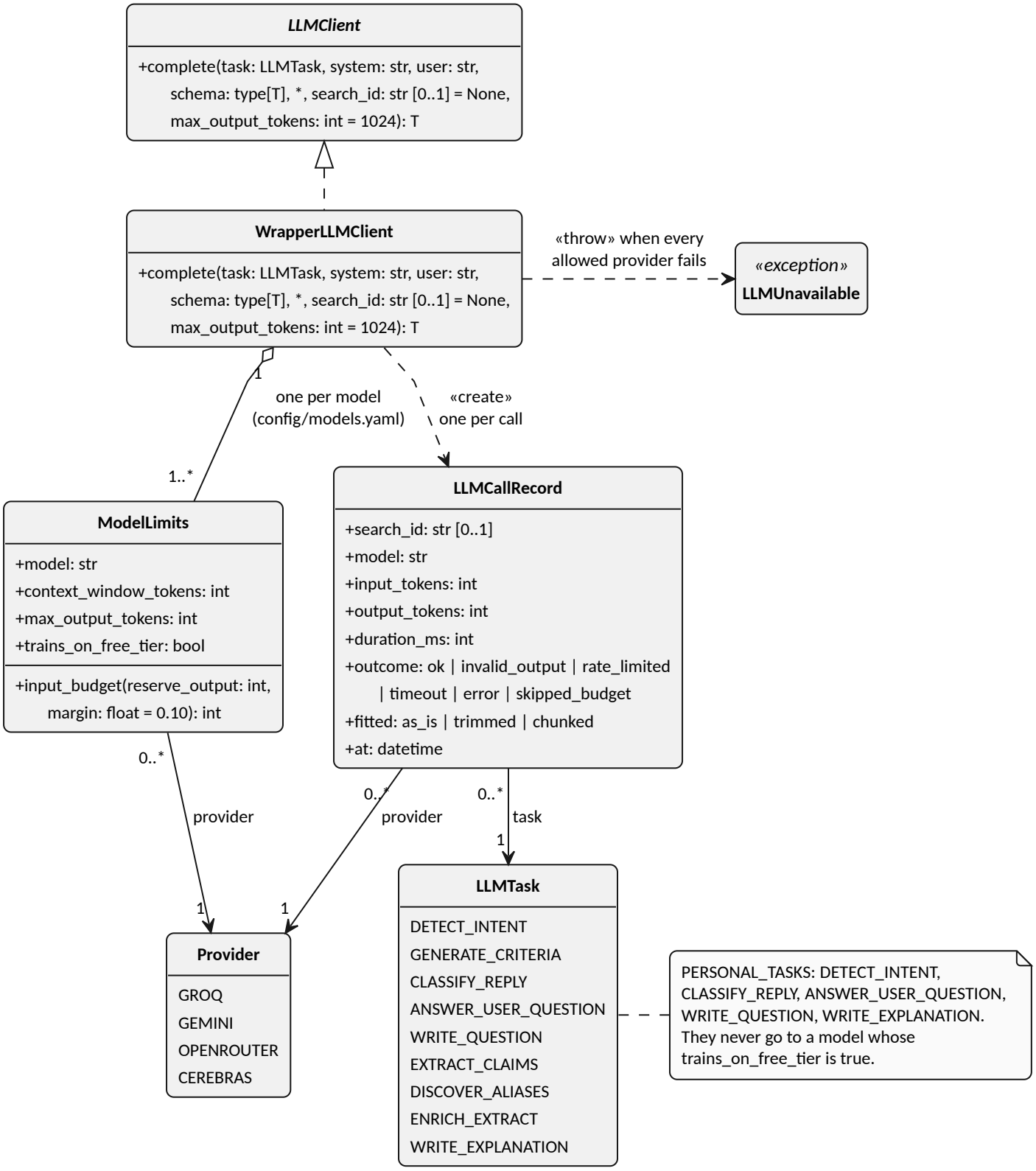


Figure 12. Class diagram: The eleven stages

Each stage implements Stage with its own input and output contract. The tagged value in braces gives the stage's fallback and its time limit from STAGE_TIME_LIMITS_S.

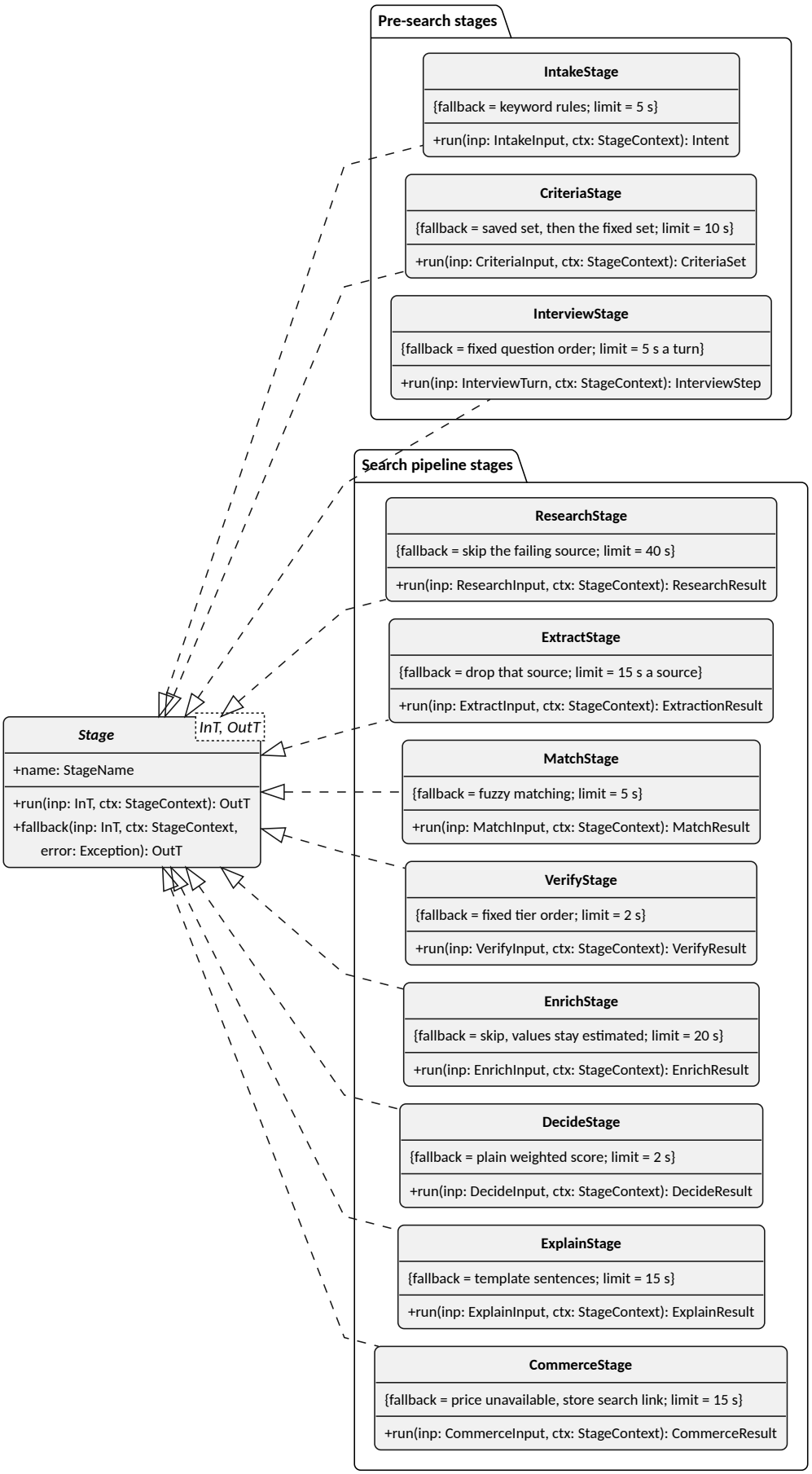


Figure 13. Class diagram: Request and criteria

Rules the contracts enforce: an Intent with no category must need clarification; category and category_label are set together; a Budget has a min, a max or both, in one currency, with min ≤ max; a CriteriaSet has unique keys and exactly one value-for-money criterion.

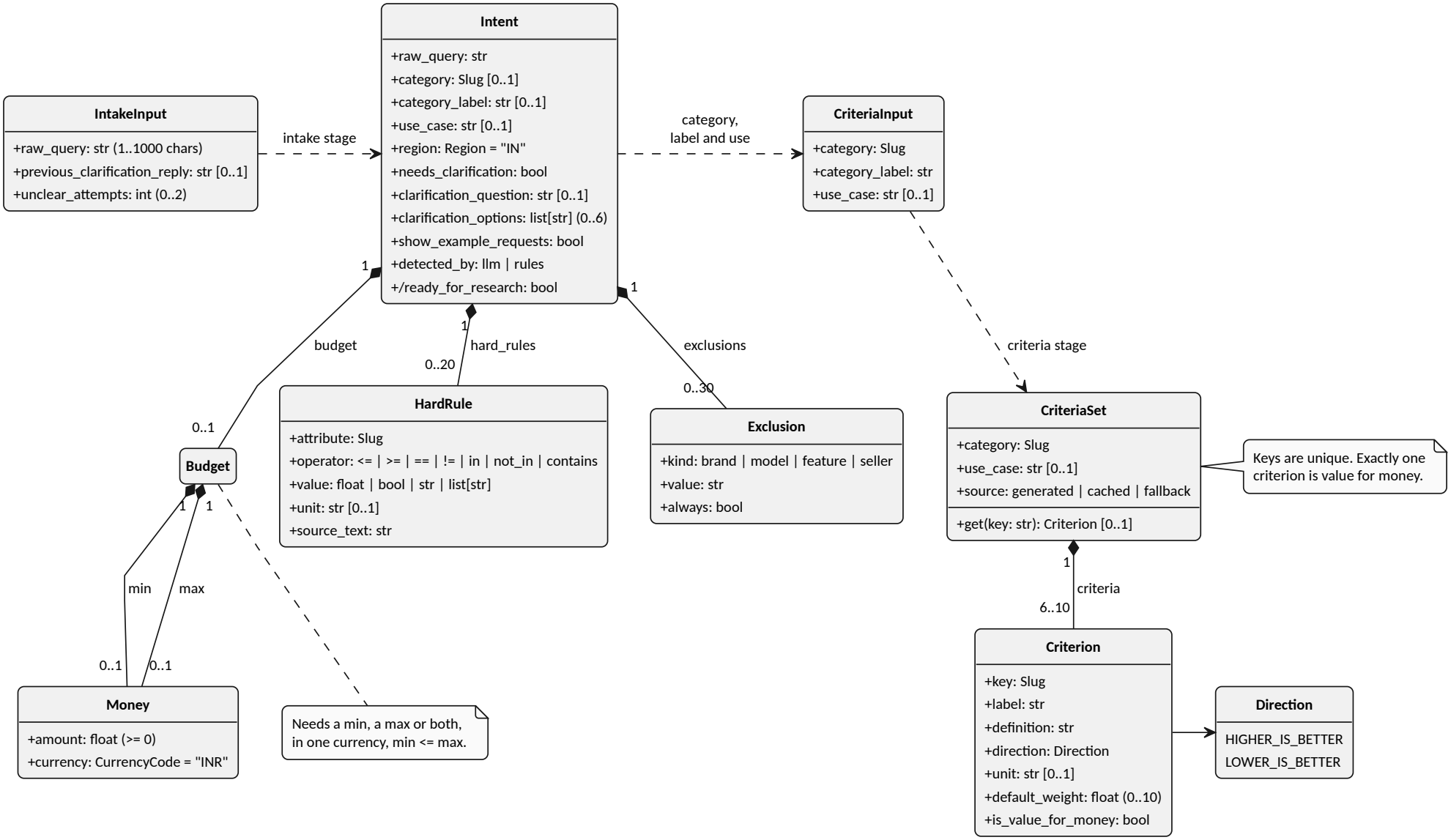


Figure 14. Class diagram: Interview and rubric

Rules: the interview starts only when the Intent is ready for research and matches the criteria's category; a finished InterviewStep carries the Rubric and no question, and every other step carries a question and no rubric; each criterion appears once in a Rubric and the normalised weights add up to 1.

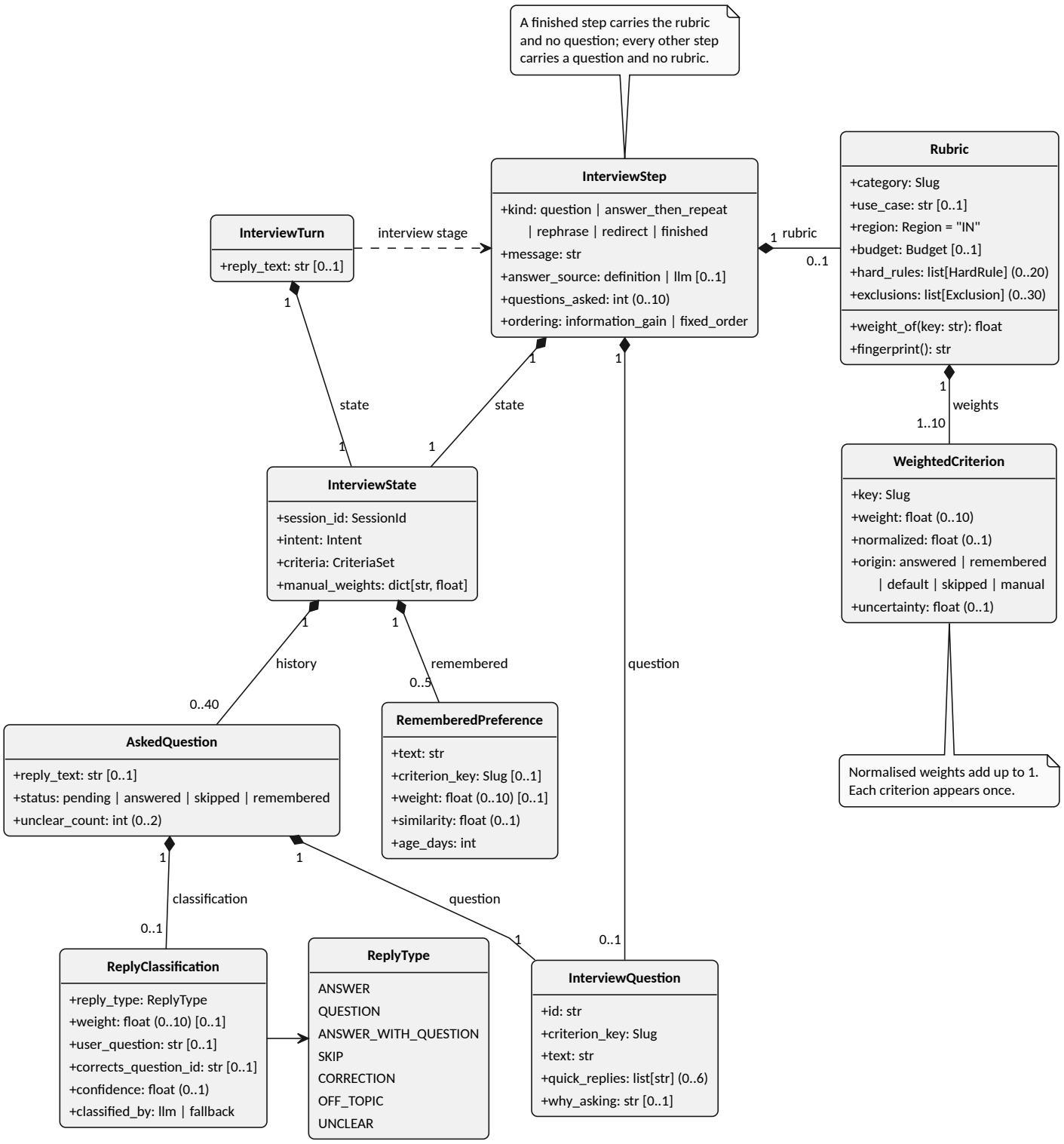


Figure 15. Class diagram: Sources, claims and products

Rules: a claim needs a value, a score or both, and a scored claim names its criterion; KNOWN and DISPUTED values cite supporting claims, and DISPUTED also cites conflicting ones; an UNKNOWN value has no score, never zero; an ESTIMATED value has a peer-average score and no claims of its own; there is one value per product and criterion.

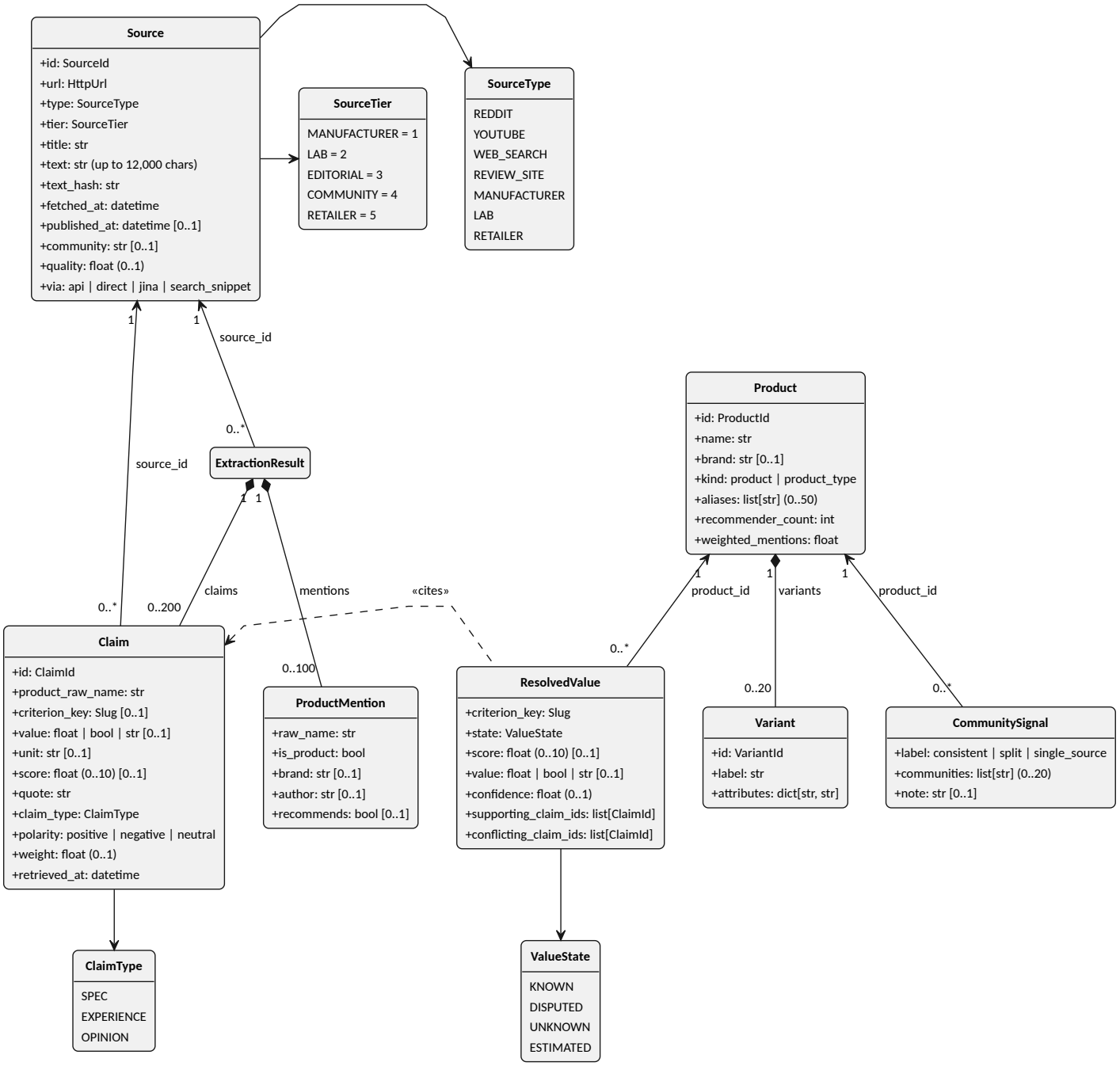


Figure 16. Class diagram: The search result

Rules: if any stage did not end OK, or the 150-second limit was hit, coverage_note must say so; every ranked product also appears in products; ranks run 1, 2, 3 in order; a product is never both ranked and filtered; an Offer is accepted only if its listing title matches the model.

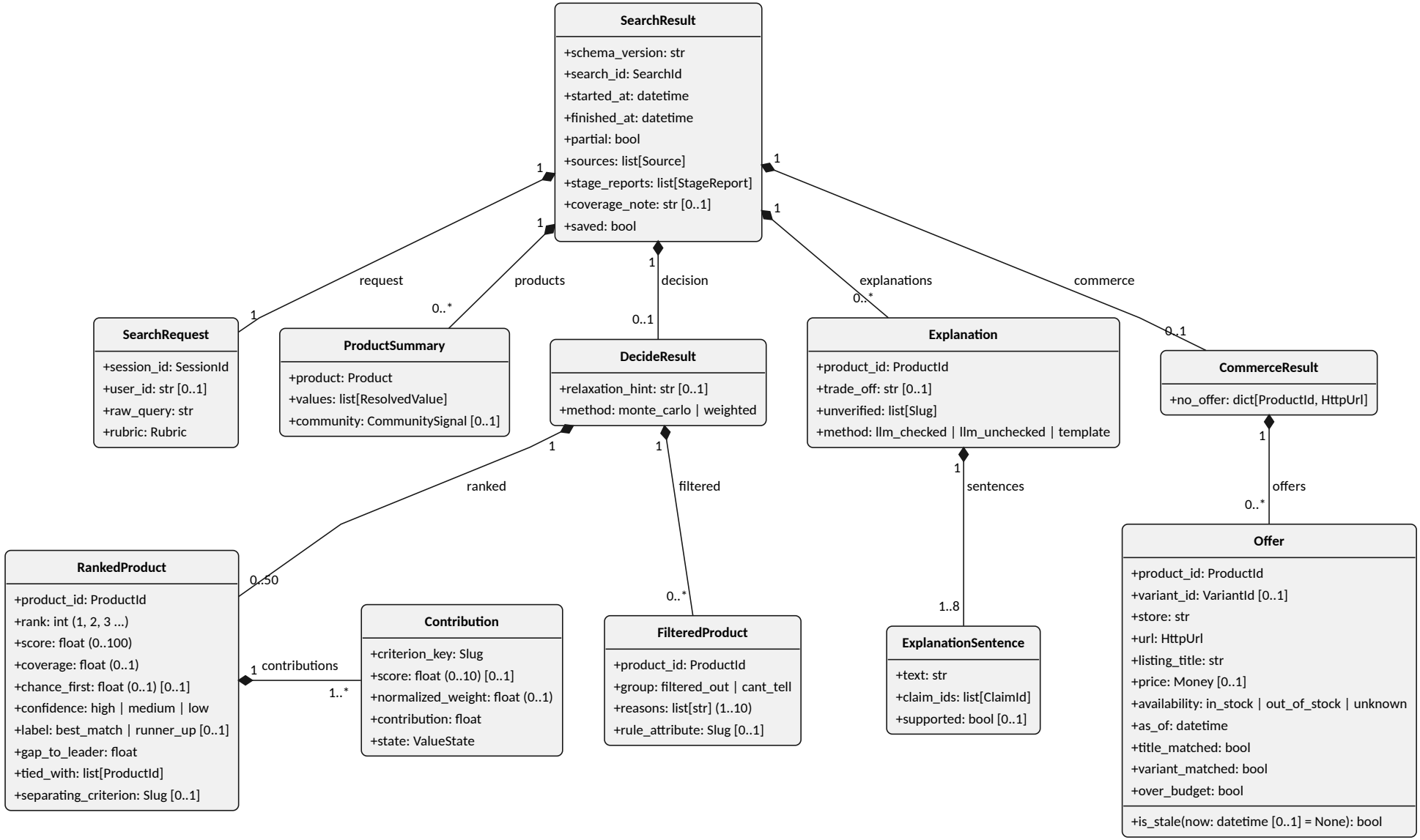


Figure 17. Sequence diagram: Detecting the request

Messages are numbered top to bottom. Solid arrows are calls, dashed arrows are replies, and the thin bars show when a participant is active. If every provider fails, IntakeStage catches LLMUnavailable and uses its keyword rules.

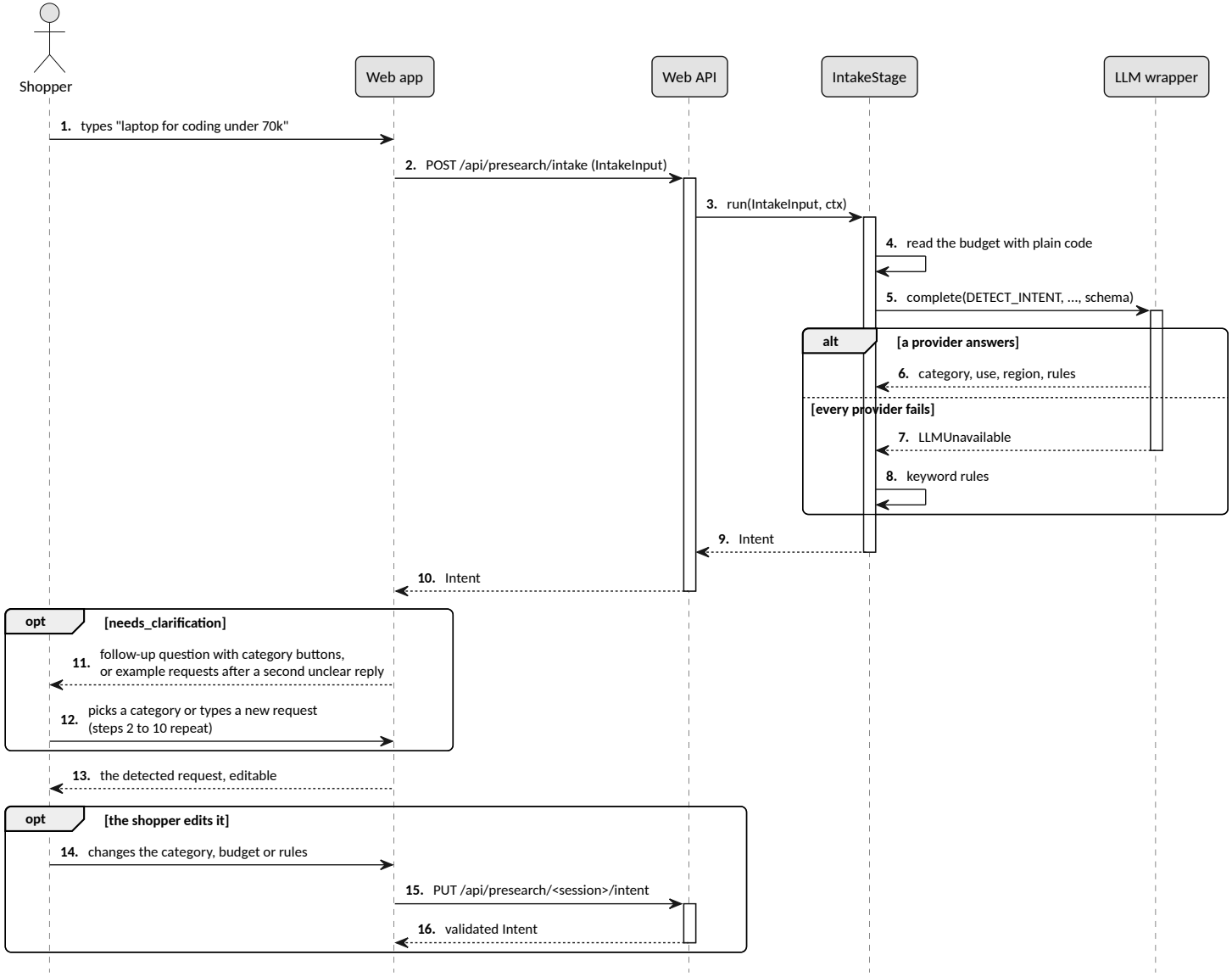


Figure 18. Sequence diagram: Preparing the criteria

Criteria are generated once per category and use, then reused. Fewer than 6 valid criteria trigger exactly one retry; if that also falls short, or no provider answers, the fixed fallback set is used.

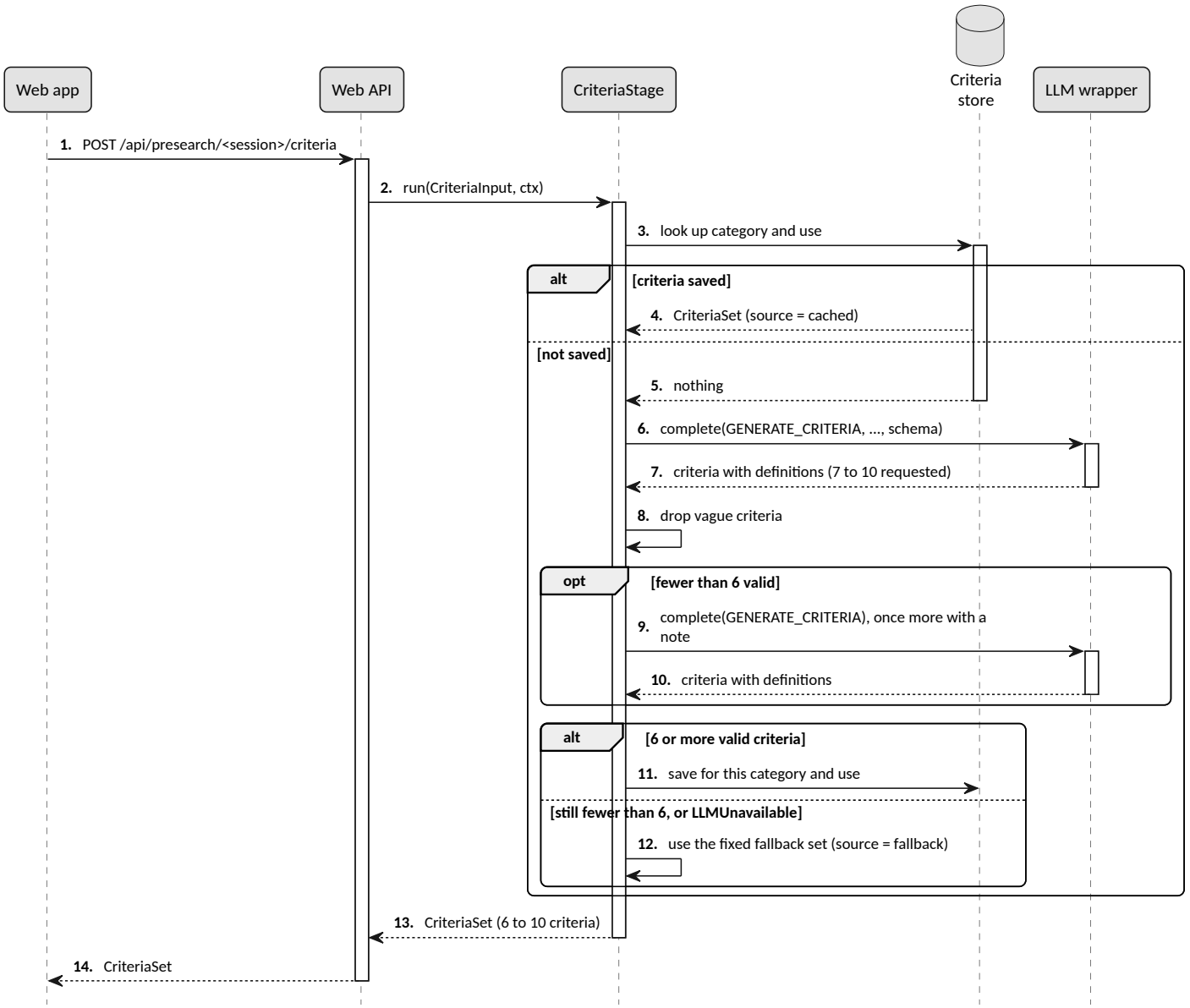


Figure 19. Sequence diagram: The interview

Arrows at the left edge are requests from the web app. The reply text goes to the LLM as data only. The loop ends with an InterviewStep of kind “finished” that carries the Rubric.

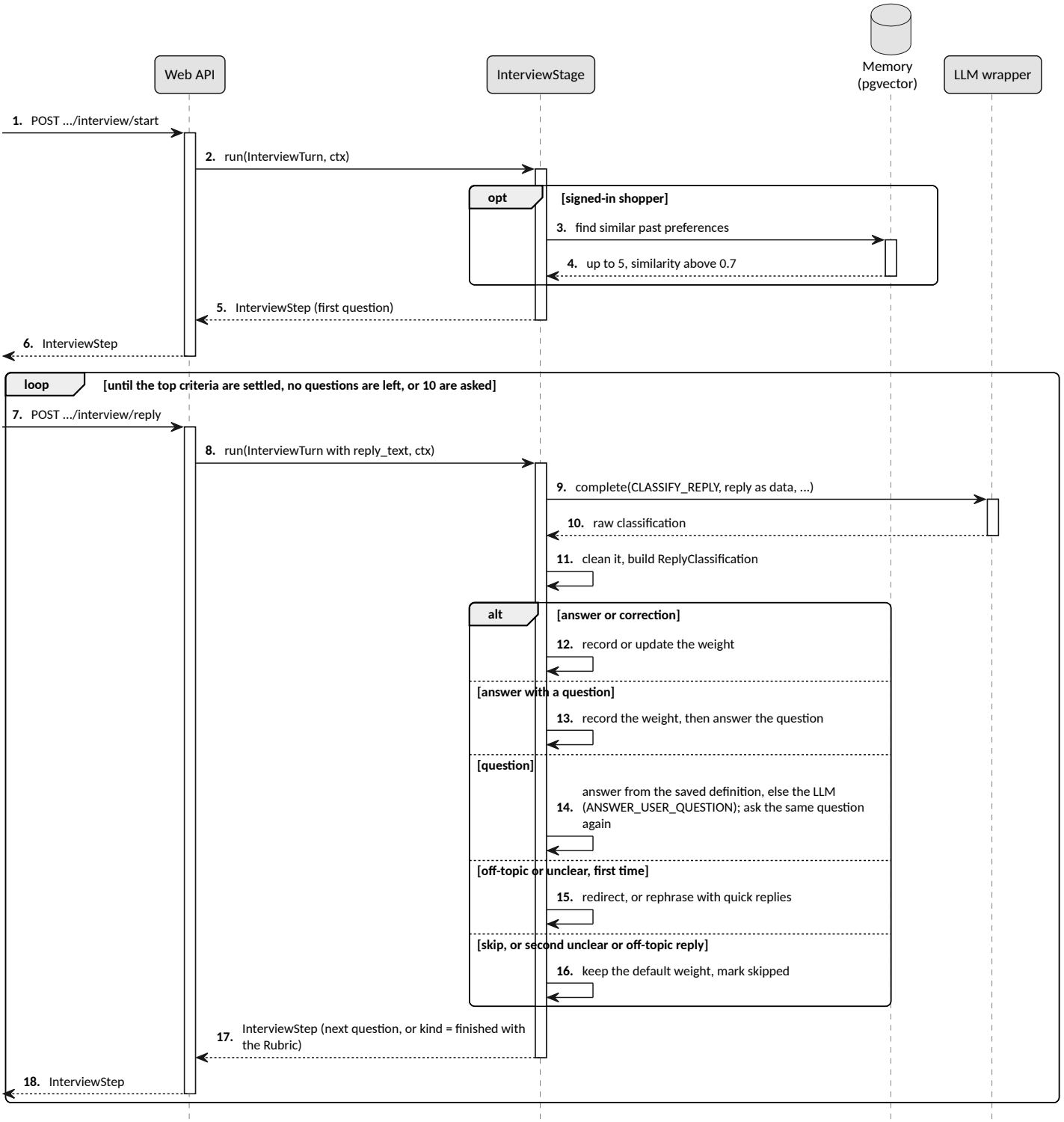


Figure 20. Sequence diagram: Search, part 1: gathering evidence

Page text goes to the LLM wrapped as data, and a failing source is skipped with a coverage note. If the cache holds a result for the same request and rubric from the last 24 hours, it is returned at once (Figure 8). Progress events reach the web app on GET /api/search/{search_id}/events.

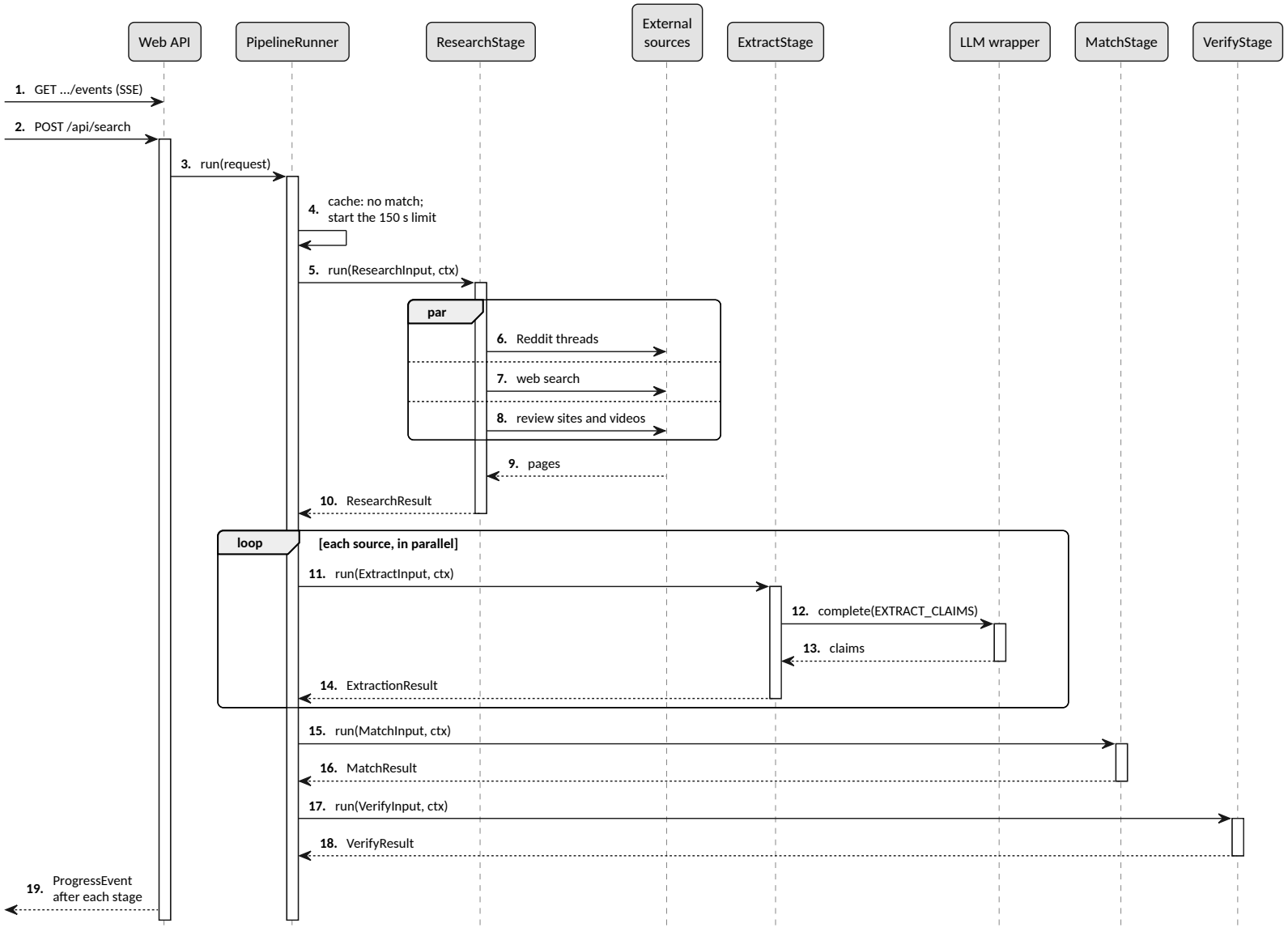


Figure 21. Sequence diagram: Search, part 2: deciding and explaining

Here ExplainStage raises when no provider answers and the runner calls fallback(), as the Stage protocol allows. DecideStage uses no LLM.

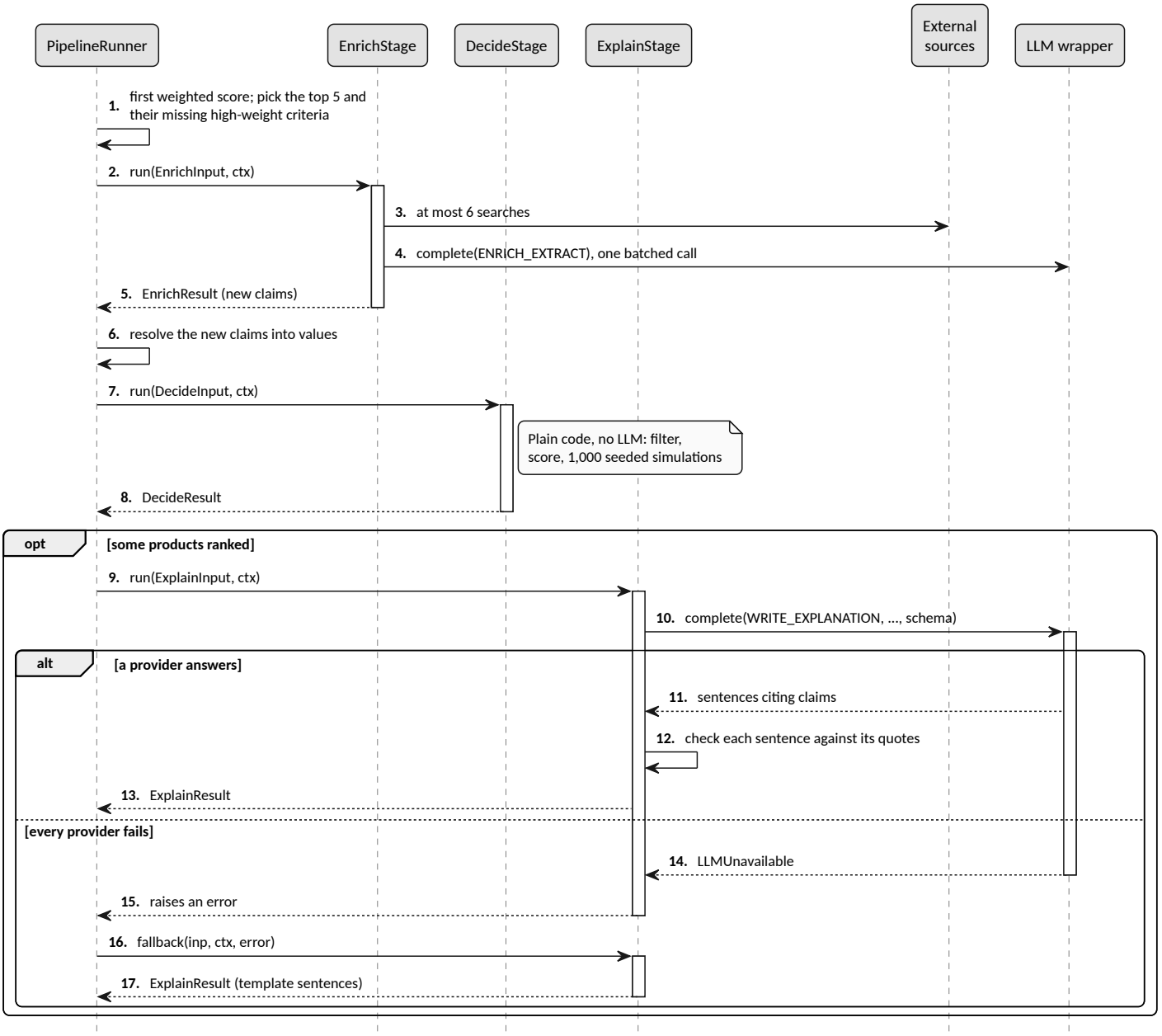


Figure 22. Sequence diagram: Search, part 3: prices, saving and the result

Offers are kept only when the listing title matches the model, and each carries its as-of time. The result is shown even if saving fails.

